

Signals of Success: Indicators of Online Product Sales – 4-Month Project Plan

Key Topics from *Hands-On Machine Learning*

- **ML Fundamentals & EDA (Ch1–2):** Understanding ML project workflow, data cleaning, feature engineering.
- **Supervised Learning (Ch4–8):** Linear/Logistic Regression (Ch4), Decision Trees (Ch7), Random Forests/Ensembles (Ch8) – for regression analysis of sales.
- **Neural Networks (Ch10–11):** Designing and training MLPs for regression (feedforward nets, optimization techniques, regularization).
- **Data Pipelines (Ch13):** Using TensorFlow's `tf.data` API to efficiently read large CSV/TFRecord datasets ¹.
- **Optional (Ch16):** RNN/LSTM for text (sentiment analysis example) if time permits.

Additional Topics to Learn

- **Network/Graph Analysis:** Review and customer-product graphs as *bipartite networks* ². Project bipartite graph to one-mode if needed (e.g. reviewer-reviewer network). Compute **bipartite clustering coefficient** (a local density measure) ³ and **eigenvector/PageRank centrality** (node influence) ⁴ for reviewers.
- **Graph Theory Methods:** Study centrality measures (Eigenvector/PageRank for importance ⁴) and clustering in two-mode networks ³. Understand how centrality might correlate with sales (see network research).
- **Time/Recency Modeling:** Learn to weight reviews by recency (e.g. time-decay functions) as a feature.
- **Natural Language Processing:** Basic sentiment analysis on review text (use pre-trained models or book's LSTM example) to extract a sentiment score.
- **Data Engineering:** Handling 10+ GB data: use **Dask** or **Spark** for out-of-core processing. Store data in efficient binary formats (Parquet or TFRecord) ⁵ to enable lazy loading and random access.

Tools & Libraries

- **Data Handling:** Python **pandas** for prototyping; **Dask** dataframes or **Apache Spark** for scalable processing (lazy evaluation) ⁵. Convert CSV to **TFRecord** or Parquet for performance.
- **Graph Analysis:** **NetworkX** (bipartite module) or **igraph** for building the reviewer-product graph and computing clustering/centrality ³ ².
- **Machine Learning:** scikit-learn (regressors, tree models), **XGBoost** (optional) for baseline models.
- **Deep Learning:** TensorFlow 2 / Keras for neural nets. Use `tf.data.Dataset` to stream data (CSV/TFRecord) during training ¹.
- **NLP/Sentiment:** NLTK/TextBlob or TensorFlow Hub (pretrained text embeddings) for extracting sentiment features.

- **Environment:** Jupyter notebooks for exploration, Git for version control, and regularly back up data samples.

Month-by-Month Timeline

• Month 1 (Weeks 1–4): Foundations & Data Preparation

- **Goals:** Grasp ML basics; explore and preprocess the Amazon review dataset (10+ GB).
- **Activities:**
 - Conduct **EDA**: schema check (reviewerID, productID, rating, helpfulness, timestamp). Visualize distributions.
 - Clean data: handle missing values, filter anomalies, and sample a manageable subset for initial experiments.
 - Compute recency: convert timestamps to “review age” or similar metric.
 - **Learning:** Read *Hands-On ML* Ch1–4 for ML workflow and linear regression fundamentals.
 - **Graph Prep:** Learn bipartite graph concept ² and represent sample data as a small reviewer–product graph (using NetworkX).
 - **Tools:** Start with pandas for dataframes; test Dask if pandas hits memory limits ⁵. Experiment with saving data in Parquet/TFRecord.
- **Deliverables:** Data profiling report (tables/charts), cleaned sample dataset, a prototype notebook showing a toy bipartite graph.
- **Progress Reporting:** Weekly email summaries; weekly meeting to discuss findings and any data issues.

• Month 2 (Weeks 5–8): Network Construction & Reviewer Scoring

- **Goals:** Build the full reviewer–product bipartite network; compute each reviewer’s “influence score.”
- **Activities:**
 - **Network Building:** Use full data to create a bipartite graph (nodes = reviewers & products; edges = reviews). Use NetworkX’s bipartite module or igraph.
 - **Reviewer Metrics:** For each reviewer, compute: (a) **Helpfulness ratio** (helpful votes/total votes), (b) **Recency ranking** (more recent = higher), (c) **Graph clustering coefficient** (local density in bipartite graph) ³, and (d) **Eigenvector/PageRank centrality** (influence score) ⁴. Combine these into a single “reviewer score.”
 - **Learning:** Study network centrality (e.g. eigenvector centrality definition ⁴, PageRank) and bipartite metrics. Review book Ch5–8 for any needed data modeling techniques.
 - **Tools:** NetworkX functions `bipartite.clustering()` and `nx.eigenvector_centrality()` (or `nx.pagerank()`) on the projected graph ². Possibly convert bipartite to one-mode graph (reviewer graph) to compute clustering.
- **Deliverables:** Table of reviewer scores; code functions for graph metrics; brief report explaining how each component of the score is computed.
- **Progress Reporting:** Mid-month demo (e.g. network graph visualizations); discuss scoring formula and preliminary results with professor.

• Month 3 (Weeks 9–12): Regression Analysis & Feature Integration

- **Goals:** Examine how the reviewer score (and other features) relate to product sales; build initial predictive models.
- **Activities:**
 - **Sales Proxy:** Determine or compute product sales/volume. If actual sales data is unavailable, use a proxy (e.g. number of reviews over a time window, or sales rank if present). Compute product “latency” (time from listing to peak sales).
 - **Additional Features:** For each product, aggregate: average rating, price (if known), total review count, and average reviewer score. Perform sentiment analysis on review texts to get a sentiment feature (e.g. average sentiment or proportion positive).
 - **Regression Modeling:** Use scikit-learn to fit models (Linear Regression, Random Forest) predicting sales/latency from the reviewer score + other features. Analyze coefficients, importance of features, R^2 , and residuals.
 - **Learning:** Deepen understanding of regression and trees from *Hands-On ML* Ch4 (Regression) and Ch7–8 (Trees, Ensembles).
 - **Tools:** pandas/Dask for feature engineering; scikit-learn for regression (`LinearRegression`, `RandomForestRegressor`). Optional: cross-validation for robust evaluation.
- **Deliverables:** Regression analysis report showing (plots of prediction vs actual, feature importances, correlation tables) and interpretation of how reviewer score correlates with sales.
- **Progress Reporting:** Present key plots (e.g. scatter of reviewer score vs sales) and model summaries to professor. Adjust approach based on feedback (e.g. refine features or score formula).

• Month 4 (Weeks 13–16): Deep Learning Model & Finalization

- **Goals:** Develop a deep-learning model that inputs reviewer score and other features (including text) to predict sales volume and latency; finalize the project.
- **Activities:**
 - **Model Design:** Construct a neural network (e.g. multi-input Keras model): one branch for numeric features (reviewer score, ratings, price, etc.), and another for text features (embedding layer for review text or aggregated text vector).
 - **Data Pipeline:** Use `tf.data` API (Ch13) to build a training pipeline that reads batched CSV or TFRecord files ¹. Include shuffling, batching, and prefetching for performance.
 - **Training/Tuning:** Train the model, using optimizers and regularization from Ch11 (e.g. dropout, learning-rate schedules). Monitor validation loss. Compare against the classical models from Month 3.
 - **Evaluation:** Compute final metrics (e.g. MAE or MSE on sales/latency), and check if reviewer-score feature is contributing.
 - **Learning:** Focus on *Hands-On ML* Ch10–11 (NN architectures, optimization) and Ch12–13 (custom training loops, data handling).
 - **Tools:** TensorFlow/Keras for model; TensorBoard for monitoring; GPUs if available for faster training.
- **Deliverables:** Final deep-learning model code (with documentation), evaluation results, and a comprehensive final report summarizing methodology, results, and future work.
- **Progress Reporting:** Schedule a final presentation or slide deck. Provide the professor with the report draft and code repository.

Learning Order and Reporting

- **Learning Sequence:** Begin with **classical ML & data skills** (Months 1–2): understanding the data, Python tools, and basic models (Ch1–4). In parallel, gradually introduce **graph theory** concepts and implement network analytics. In Month 3, add more advanced models (tree ensembles Ch7–8) and experiment with features. Finally, Month 4 is devoted to **deep learning** (Ch10–13) once a solid foundation is built.
- **Progress Checks:** Plan weekly check-ins or short reports (via email or lab meeting) to discuss challenges and intermediate findings. Use version control (Git) and Jupyter notebooks to track work. Provide **monthly summaries** to the professor highlighting completed tasks and next goals.

Key References (for methods): The review network is modeled as a bipartite graph ², so we will apply graph metrics: clustering coefficient (density) ³ and eigenvector/PageRank centrality (node influence) ⁴. For handling the large dataset, we will use scalable data tools and TensorFlow's Data API to read CSV/TFRecord files efficiently ¹ ⁵. These methods, combined with the topics from the textbook, will guide our workflow.

¹ Hands-on Machine Learning with Scikit-Learn, Keras & TensorFlow

<https://studylib.net/doc/27094611/aurelien-geron-hands-on-machine-learning-with-scikit-lear...>

² Rating and perceived helpfulness in a bipartite network of online product reviews | Electronic Commerce Research

<https://link.springer.com/article/10.1007/s10660-023-09725-1>

³ clustering — NetworkX 3.6.1 documentation

<https://networkx.org/documentation/stable/reference/algorithms/generated/networkx.algorithms.bipartite.cluster.clustering.html>

⁴ Eigenvector centrality - Wikipedia

https://en.wikipedia.org/wiki/Eigenvector_centrality

⁵ Dask Best Practices — Dask documentation

<https://docs.dask.org/en/stable/best-practices.html>